



Neo_street UML Conception

👤 Créée par	🔒 SnoXx
🕒 Date de création	@11 mars 2026 12:42
☰ Catégorie	doc
👤 Dernière modification par	🔒 SnoXx
🕒 Heure de la dernière modification	@11 mars 2026 12:43

Conception Technique : Application E-Commerce

1. Introduction

Ce document décrit les choix architecturaux retenus pour le développement de notre plateforme de e-commerce. L'objectif est de garantir une application **évolutive, maintenable et cohérente** avec les besoins métier.

2. Architecture Globale

L'application repose sur une **architecture en couches (n-tier)**, séparant strictement la logique métier du stockage et de l'interface utilisateur.

- **Couche Présentation (Frontend)** : Responsable de l'affichage et de l'expérience utilisateur. Elle consomme les ressources fournies par l'API.
- **Couche Service (Backend/API)** : Point central de l'intelligence applicative. Elle orchestre les interactions entre les utilisateurs, la gestion des produits et le panier.
- **Couche Persistance (Données)** : Encapsulée derrière un service de stockage pour garantir l'intégrité des données et faciliter les futures migrations de base de données.

3. Choix Modélisation et POO

Le diagramme de classe a été conçu selon les principes de la **Programmation Orientée Objet (POO)** :

- **Encapsulation** : Les attributs sont privés () pour assurer la sécurité des données.
- **Modularité** : L'utilisation d'une **classe d'association** (`CartProduct`) permet de gérer la relation "many-to-many" entre `Product` et `Cart` tout en portant la donnée métier `quantity`.
- **Typage** : Le choix de `float` pour le prix garantit la précision des calculs financiers, contrairement au format `str`.

4. Flux d'interaction (Diagramme de Séquence)

L'interaction "Ajout au panier" illustre le comportement synchrone du système :

1. Le **Frontend** envoie une requête HTTP à l'**API**.
2. Le **Contrôleur** valide la demande en interrogeant le service métier.
3. Les données sont persistées via le **Service de Stockage**.
4. Une confirmation est renvoyée au client final, garantissant une mise à jour fluide de l'état du panier.

5. Justification de la décomposition en composants

Le choix de découper le backend en composants distincts (`API` , `Gestionnaire de Panier` , `Service de Stockage`) répond à deux impératifs :

- **Isolation des responsabilités** : Le `Gestionnaire de Panier` ne se soucie pas de *comment* les données sont stockées, mais seulement *quelles* données enregistrer.
- **Facilité de maintenance** : Si la logique de calcul du panier doit évoluer (ajout de remises, codes promos), seule la brique `Gestionnaire de Panier` est modifiée, sans impacter le reste du système.